DATABASE DESIGN AND IMPLEMENTATION REPORT

- **E-Commerce Platform**

1. Problem Statement

The Mini E-Commerce Platform manages users, products, categories, orders, and roles. It allows customers to browse items by category, place orders, and manage their profiles. The system tracks inventory levels and user-specific data like multiple contact numbers and roles, ensuring high data integrity and optimized query performance through a normalized relational structure.

2. Entities & Attributes

- **Users**: user_id, full_name, email, created_at
- **User_Phones**: phone_id, user_id, phone_number
- **User_Profiles**: profile_id, user_id, address, bio
- **Categories**: category_id, category_name
- **Products**: product_id, category_id, name, price, stock_quantity
- **Orders**: order_id, user_id, order_date, status
- **Order_Items**: order_item_id, order_id, product_id, quantity, unit_price
- **User_Roles_Mapping**: user_id, role_id

3. Relationships

Users to User_Phones: One-to-Many (1:N) – A user can have multiple contact numbers.

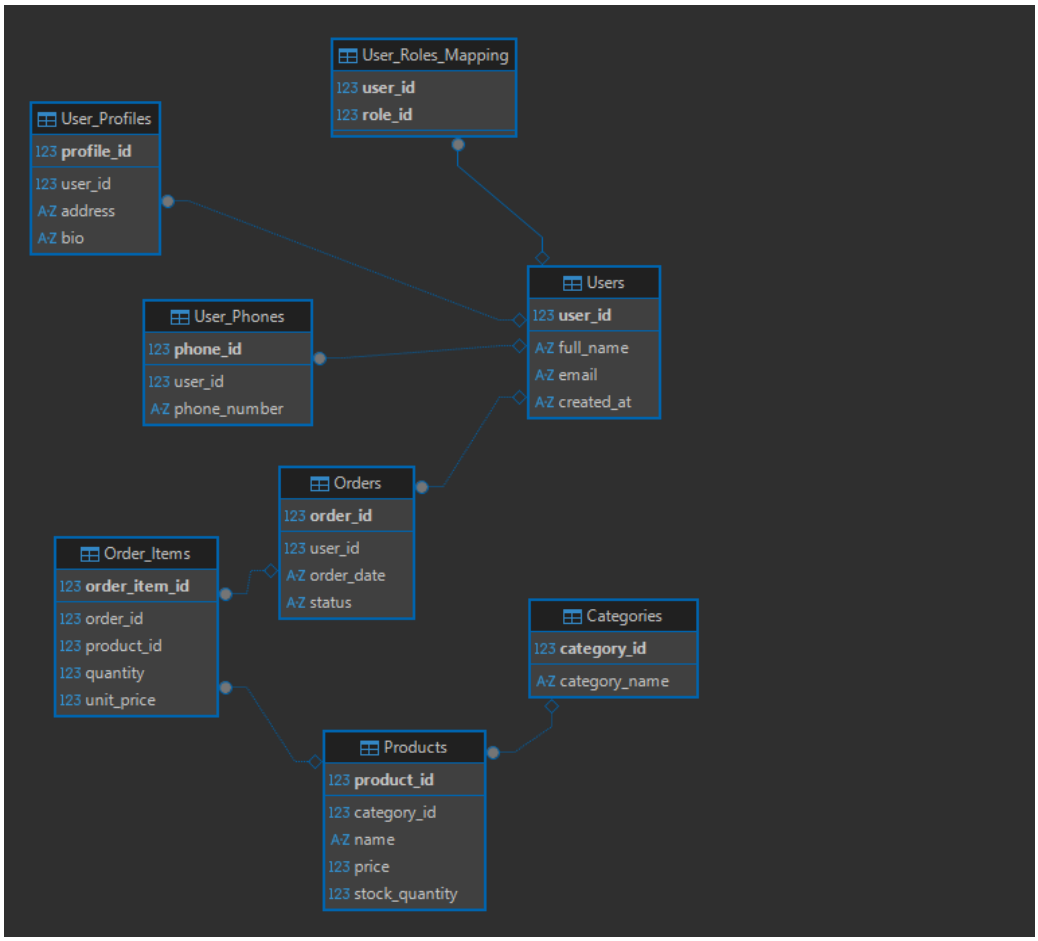
Users to User_Profiles: One-to-One (1:1) – Each user has one unique detailed profile.

Categories to Products: One-to-Many (1:N) – One category contains multiple products.

Orders to Products: Many-to-Many (M:N) – Resolved via the **Order_Items** junction table.

Users to Roles: Many-to-Many (M:N) – Resolved via the **User_Roles_Mapping** table.

4. ER Diagram



The design follows a central-hub architecture where the Users table acts as the primary anchor, connecting to User_Phones, User_Profiles, and User_Roles_Mapping to separate contact, metadata, and permission data. The transactional logic is isolated within the Orders and Order_Items tables, which bridge the relationship between customers and the categorized Products inventory.

5. Normalization

- **1NF**: All attributes contain atomic values. Multi-valued attributes like phone numbers are moved to a separate table (User_Phones).
- **2NF**: All tables are in 1NF and all non-key attributes are fully functionally dependent on the primary key.
- **3NF**: Transitive dependencies are eliminated. For example, product categories are stored in Categories rather than Products to prevent redundancy.

6. Relationship Handling

One-to-Many relationships are implemented using Foreign Keys (e.g., user_id in Orders). The Many-to-Many relationship between Orders and Products is handled by Order_Items, which stores specific transaction details like quantity and unit price at the time of purchase.

7. Table Schema & Implementation

1. Users

```
CREATE TABLE Users(
user_id INTEGER PRIMARY KEY AUTOINCREMENT,
full_name VARCHAR(100) NOT NULL,
email VARCHAR(150) UNIQUE NOT NULL,
created_at TIMESTAMP
);
```

2. User_Phones

```
CREATE TABLE User_Phones(
phone_id INTEGER PRIMARY KEY AUTOINCREMENT,
user_id INTEGER,
phone_number VARCHAR(15) NOT NULL,
FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```

3. User_Profiles

```
CREATE TABLE User_Profiles(
profile_id INTEGER PRIMARY KEY AUTOINCREMENT,
user_id INTEGER UNIQUE,
address TEXT,
bio TEXT,
FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```

4. Categories

```
CREATE TABLE Categories(
```

```
category_id INTEGER PRIMARY KEY AUTOINCREMENT,
category_name VARCHAR(100) NOT NULL
);
```

5. Products

```
CREATE TABLE Products(
product_id INTEGER PRIMARY KEY AUTOINCREMENT,
category_id INTEGER,
name VARCHAR(200) NOT NULL,
price DECIMAL(10, 2) NOT NULL,
stock_quantity INTEGER DEFAULT 0,
FOREIGN KEY (category_id) REFERENCES Categories(category_id)
);
```

6. Orders

```
CREATE TABLE Orders(
order_id INTEGER PRIMARY KEY AUTOINCREMENT,
user_id INTEGER,
order_date TIMESTAMP,
status VARCHAR(50),
FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```

7. Order_Items

```
CREATE TABLE Order_Items(
order_item_id INTEGER PRIMARY KEY AUTOINCREMENT,
order_id INTEGER,
product_id INTEGER,
quantity INTEGER NOT NULL,
unit_price DECIMAL(10, 2) NOT NULL,
FOREIGN KEY (order_id) REFERENCES Orders(order_id),
```

```
FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
```

8. User_Roles_Mapping

```
CREATE TABLE User_Roles_Mapping(
user_id INTEGER PRIMARY KEY,
role_id INTEGER,
FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```

➢ College Campus Hostel Booking

1. Problem Statement

The Campus Hostel Booking system automates the management of hostel infrastructure (Blocks and Rooms), student residents, and facility usage. It tracks student room allocations by academic year, manages recurring amenity costs, and handles event registrations. The system provides insights into revenue generation and occupancy rates across different hostel blocks.

2. Entities & Attributes

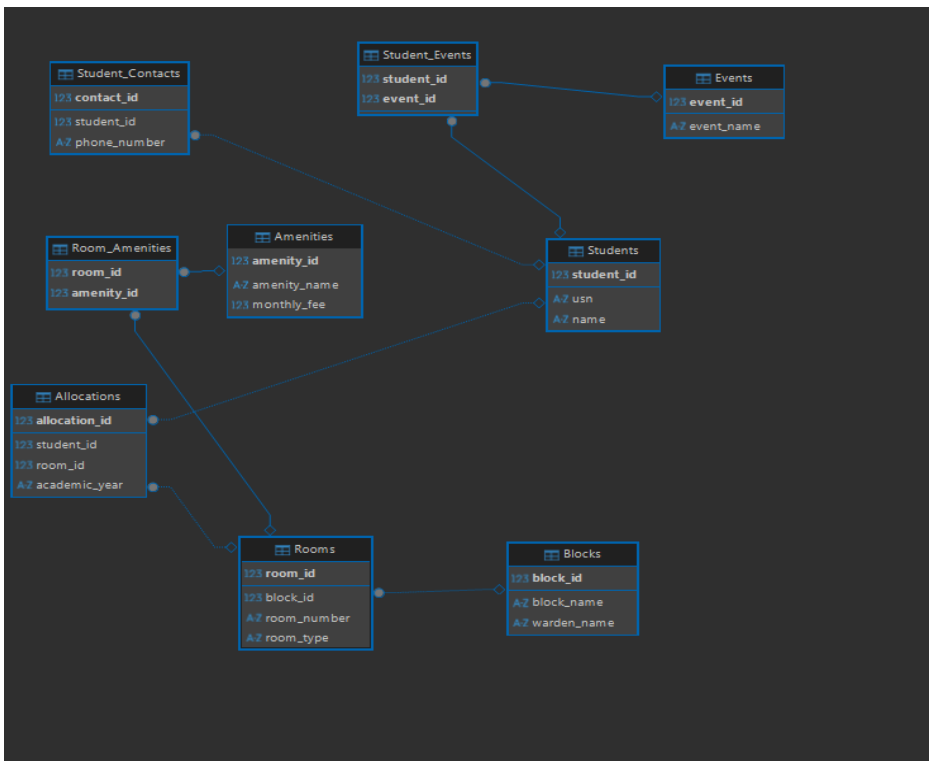
- **Blocks**: block_id, block_name, warden_name
- **Rooms**: room_id, block_id, room_number, room_type
- **Students**: student_id, usn, name
- **Student_Contacts**: contact_id, student_id, phone_number
- **Allocations**: allocation_id, student_id, room_id, academic_year
- **Amenities**: amenity_id, amenity_name, monthly_fee
- **Room_Amenities**: room_id, amenity_id
- **Events**: event_id, event_name

- **Student_Events**: student_id, event_id

3. Relationships

- **Blocks to Rooms**: One-to-Many (1:N) – A hostel block contains several rooms.
- **Students to Rooms**: Many-to-Many (M:N) – Resolved via the **Allocations** table to track residence over different academic years.
- **Rooms to Amenities**: Many-to-Many (M:N) – Resolved via the **Room_Amenities** table to track which facilities are available in which room.
- **Students to Events**: Many-to-Many (M:N) – Resolved via the **Student_Events** junction table.

4. ER Diagram



The design follows a central-hub architecture where the Students table serves as the core entity, linking to Student_Contacts for multi-valued data and Allocations to manage residence history. The operational logic is handled by the Rooms and Room_Amenities link to Amenities, while a separate many-to-many bridge between Students and Events manages extracurricular participation.

5. Normalization

- **1NF**: Tables like Student_Contacts ensure that no student has multiple phone numbers in a single column.
- **2NF**: All non-prime attributes are fully dependent on the PK.
- **3NF**: Data is split so that Warden information is tied to a Block rather than repeated for every Student or Room.

6. Relationship Handling

The Many-to-Many relationship for room facilities is handled via Room_Amenities, allowing for flexible billing. The Multi-valued attribute of student contacts is handled via Student_Contacts. CHECK constraints ensure only valid room types ('Single', 'Double', 'Triple') are accepted.

7. Table Schema & Implementation

1. Blocks

```
CREATE TABLE Blocks(
block_id INTEGER PRIMARY KEY AUTOINCREMENT,
block_name VARCHAR(50) NOT NULL,
warden_name VARCHAR(100)
);
```

2. Rooms

```
CREATE TABLE Rooms(
room_id INTEGER PRIMARY KEY AUTOINCREMENT,
block_id INTEGER,
room_number VARCHAR(10) NOT NULL,
room_type VARCHAR(50),
FOREIGN KEY (block_id) REFERENCES Blocks(block_id)
);
```

3. Students

```
CREATE TABLE Students(
```

```
student_id INTEGER PRIMARY KEY AUTOINCREMENT,
usn VARCHAR(20) UNIQUE NOT NULL,
name VARCHAR(100) NOT NULL
);
```

4. Student_Contacts

```
CREATE TABLE Student_Contacts(
contact_id INTEGER PRIMARY KEY AUTOINCREMENT,
student_id INTEGER,
phone_number VARCHAR(15) NOT NULL,
FOREIGN KEY (student_id) REFERENCES Students(student_id)
);
```

5. Allocations

```
CREATE TABLE Allocations(
allocation_id INTEGER PRIMARY KEY AUTOINCREMENT,
student_id INTEGER,
room_id INTEGER,
academic_year VARCHAR(10),
FOREIGN KEY (student_id) REFERENCES Students(student_id),
FOREIGN KEY (room_id) REFERENCES Rooms(room_id)
);
```

6. Amenities

```
CREATE TABLE Amenities(
amenity_id INTEGER PRIMARY KEY AUTOINCREMENT,
amenity_name VARCHAR(50) NOT NULL,
monthly_fee DECIMAL(10, 2)
);
```

7. Room_Amenities

```
CREATE TABLE Room_Amenities(
```

```
room_id INTEGER,
amenity_id INTEGER,
PRIMARY KEY (room_id, amenity_id),
FOREIGN KEY (room_id) REFERENCES Rooms(room_id),
FOREIGN KEY (amenity_id) REFERENCES Amenities(amenity_id)
);
```

8. Events

```
CREATE TABLE Events(
event_id INTEGER PRIMARY KEY AUTOINCREMENT,
event_name VARCHAR(100) NOT NULL
);
```

9. Student_Events

```
CREATE TABLE Student_Events(
student_id INTEGER,
event_id INTEGER,
PRIMARY KEY (student_id, event_id),
FOREIGN KEY (student_id) REFERENCES Students(student_id),
FOREIGN KEY (event_id) REFERENCES Events(event_id)
);
```